Université de Limoges

NULL POUR LES NULS

Différentes manières de traiter un null en Java

Contexte : Manipulation de données (ex: fichiers CSV, champs facultatifs).

Le problème : Oubli de vérification -> **NullPointerException** et plantage net de l'application.

Par Sacha, Yassine et Nathan

SOMMAIRE

1. Définition : Qu'est-ce que null et la NullPointerException ?

2. Traitements historiques : Vérification défensive, propagation (throws) et capture (try...catch).

3. Alternative moderne : Optional<T> (Exemple du Jeu de Dés).

4. Approfondissement : Mockito, Objects.requireNonNull et les Records.

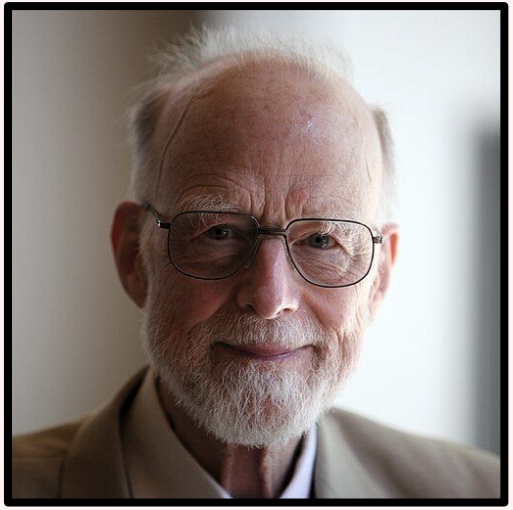
5. Conclusion & Pratique: Pattern Null Object, Live-coding et QCM.

Qu'est-ce que null et la NullPointerException ?

Définition : Référence vide, absence de valeur en mémoire.
L'historique : Le « billion-dollar mistake » (Tony Hoare, 1965).

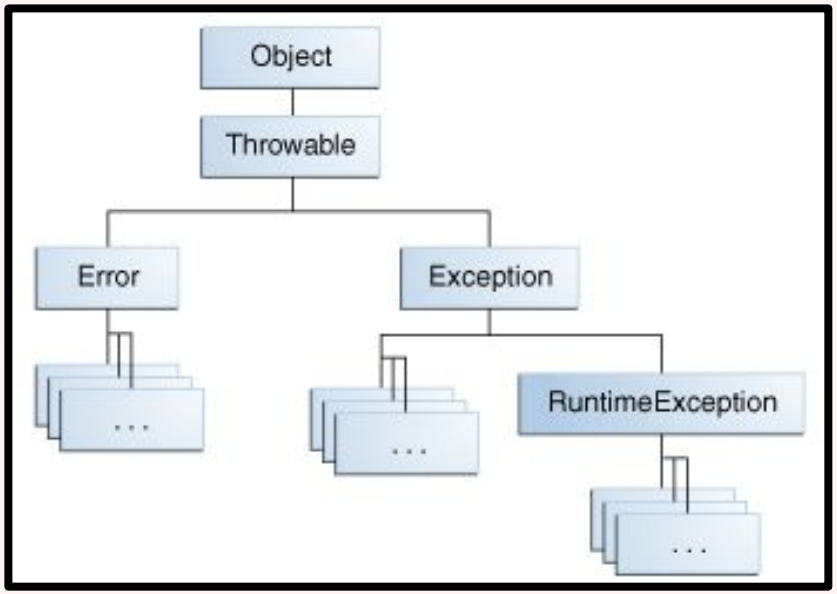
Classification des erreurs en Java :

Erreurs de **compilation** vs Erreurs à **l'exécution** (JVM).
Error (fatales/internes) vs **Exception** (récupérables).



Hoare en 2011

Qu'est-ce que null et la NullPointerException ?



Classe Throwable

La **NullPointerException :**

Appartient aux **RuntimeException (Unchecked Exceptions).**

Non contrôlée par le compilateur : code compilable mais plantage à l'exécution.

Les différentes manières de traiter un null en Java

La vérification défensive (if (x != null)) :

Simple mais conduit à des structures imbriquées complexes.

```
if (utilisateur != null) {
    Adresse adr = utilisateur.getAdresse();
    if (adr != null) {
        System.out.println(adr.getVille());
    }
}
```

1. Le déclenchement (throw new)

On lance l'exception

```
if (utilisateur == null) {
    throw new
IllegalArgumentException("L'utilisateur
ne peut pas être null !");
}
```

Les différentes manières de traiter un null en Java

2. La propagation (throws) :

Délègue le traitement à la méthode appelante.

Jamais de throws dans le main (dernier rempart pour attraper les exceptions).

3. La capture (try...catch) :

Isole le code à risque, intercepte l'erreur et afficher l'erreur via **e.getMessage().**

```
public void verifierDonnees() throws
IllegalArgumentException {
    // Si une exception survient ici,
elle est propagée au niveau supérieur
}
```

```
try {
    recupererVille(utilisateur);
} catch (IllegalArgumentException e)
{
    System.out.println("Erreur gérée
: " + e.getMessage());
}
```

Cependant, une alternative moderne existe

L'alternative moderne : Optional<T> (Java8)

Principe de java.util.Optional<T> : Un conteneur qui encapsule soit une valeur, soit rien (Optional.empty()).

Objectif : Éviter l'utilisation explicite de null et éliminer les risques de NullPointerException.

Avantage : Remplace les valeurs magiques (magic numbers comme -1) et indique clairement l'absence potentielle d'une donnée.

Méthodes clés de la Javadoc :
Création : Optional.empty(), Optional.of(), Optional.ofNullable()
Extraction : isPresent(), get(), orElse()

Optional<T> : un exemple concret

Cas courant : la recherche d'un utilisateur en base de données.

Avant : la méthode renvoyait null si l'utilisateur n'existait pas → plantage si on oubliait de vérifier.

Avec Optional : le type de retour annonce que le résultat peut être absent. L'appelant est forcé de gérer ce cas (programmation défensive).

```
public Optional<Utilisateur>
    findUtilisateurById(int id) {
    Utilisateur user =
        database.get(id);
    return Optional.ofNullable(user);
}
```

Optional dans le Kata Dice (Mockito)

Classe Player du TP Mockito.

Avant : lastValue valait -1 (valeur magique) pour dire « aucun lancer ».

Maintenant : Optional<Integer> lastValue

- Optional.empty() à la création
- Optional.of(dice.roll()) après un lancer

```java
import java.util.Optional;

public class Player {

    private String name;
    private Optional<Integer> lastValue;

    public Player(String name) {
        this.name = name;
        this.lastValue = Optional.empty();
    }

    public void play(Dice dice) {
        this.lastValue = Optional.of(dice.roll());
    }

    public Optional<Integer> getLastValue() {
        return lastValue;
    }

    public String getName() {
        return name;
    }
}
```

Tests unitaires avec Optional

Avant : assertion sur une valeur arbitraire comme -1.

Maintenant : on vérifie explicitement si la valeur est présente ou non (isPresent()). L'intention du test devient limpide.

```java
import static org.junit.jupiter.api.Assertions.assertFalse;

class PlayerTest {

    @Test
    void playerWithoutPlay_shouldHaveEmptyLastValue() {
        Player player = new Player("Bob");

        assertFalse(player.getLastValue().isPresent());
    }
}
```

La méthode statique Objects.requireNonNull

Optional : parfait pour les valeurs de retour, pas pour vérifier un paramètre.

requireNonNull(T obj, String message) : lève une NullPointerException avec le message si obj est null, sinon retourne obj.

Principe Fail-Fast : bloquer le problème dès l'entrée du constructeur, plutôt qu'un plantage difficile à déboguer plus loin.

```java
import java.util.Objects;

public class Utilisateur {

    private String pseudo;

    public Utilisateur(String pseudo) {
        this.pseudo = Objects.requireNonNull(pseudo, "Le pseudo ne peut pas être null");
    }
}
```

requireNonNull et les records

Record : classe de données immuable, syntaxe concise (constructeur, accesseurs, equals, hashCode et toString générés).

Constructeur compact : ajoute la validation sans réécrire les affectations.

Résultat : immuabilité + validation dès la création. Un DTO ne transporte jamais de références nulles illégales.

```java
import java.util.Objects;

public record UtilisateurRecord(String pseudo, String email) {

    public UtilisateurRecord {
        Objects.requireNonNull(pseudo, "Le pseudo est obligatoire");
        Objects.requireNonNull(email, "L'email est obligatoire");

    }
}
```

Null Object : le besoin

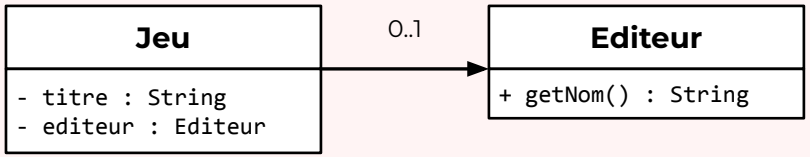
Exemple fil rouge : le fichier CSV de ventes de jeux vidéo.

Besoin : afficher le nom de l'éditeur de chaque jeu. Ce champ est facultatif.

Modélisation de départ : l'attribut editeur vaut null quand il n'est pas renseigné.

```java
public class Jeu {
    private String titre;
    private Editeur editeur; // null si absent
    public Editeur getEditeur() {
        return editeur;
    }
}

// Code client
for (Jeu jeu : jeux) {
    if (jeu.getEditeur() != null) {
        println(jeu.getEditeur().getNom());
    } else {
        println("éditeur inconnu");
    }
}
```



Les problèmes de cette modélisation

Test dupliqué
Le if (x != null) est répété partout où l'on utilise l'éditeur. Un seul oubli : NullPointerException.

Code client pollué
Il gère un cas particulier qui n'est pas son rôle.

Comportement incohérent
Chaque développeur décide seul quoi faire quand l'éditeur est absent.

Idée : ne plus manipuler null, mais un vrai objet qui respecte le même contrat, avec un comportement neutre.

La solution : le pattern Null Object

```java
public interface Editeur {
    String getNom();
}

public class EditeurReel implements Editeur {
    private final String nom;
    public EditeurReel(String nom) { this.nom = nom; }
    @Override
    public String getNom() { return nom; }
}

public class EditeurInconnu implements Editeur {
    @Override
    public String getNom() { return "Inconnu"; }
}
```

Avant

```java
if (jeu.getEditeur() != null) {
    println(jeu.getEditeur().getNom());
} else {
    println("éditeur inconnu");
}
```

Après

```java
println(jeu.getEditeur().getNom());
```

Plus de if, plus de NullPointerException : le cas « absent » est géré dans une seule classe.

Null Object : nom, type et intention

Nom : Null Object (ou « Special Case » chez Martin Fowler)

Type : pattern de conception, mais pas un pattern du GoF

Problématique : comment éviter de multiplier les tests null dans le code client et traiter l'absence d'objet de façon cohérente ?

INTENTION

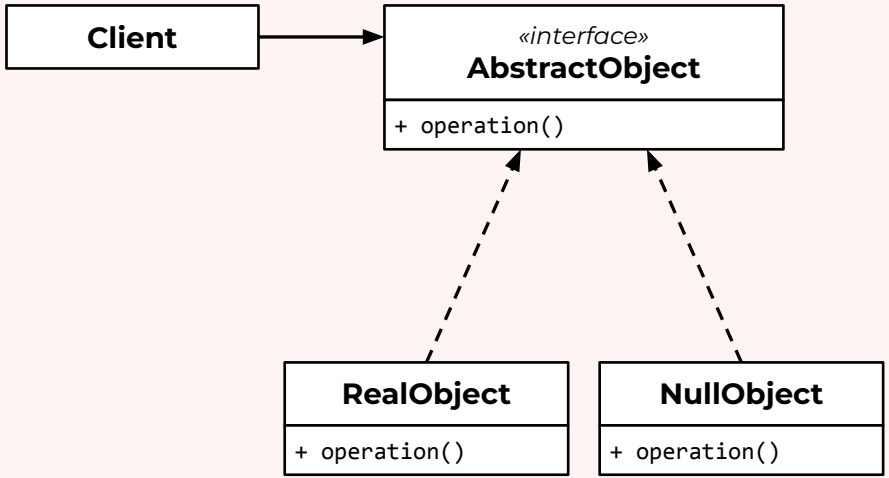
Fournir un objet de substitution qui implémente la même interface que les objets réels, avec un comportement neutre, pour remplacer la valeur null.

Null Object : les classes participantes

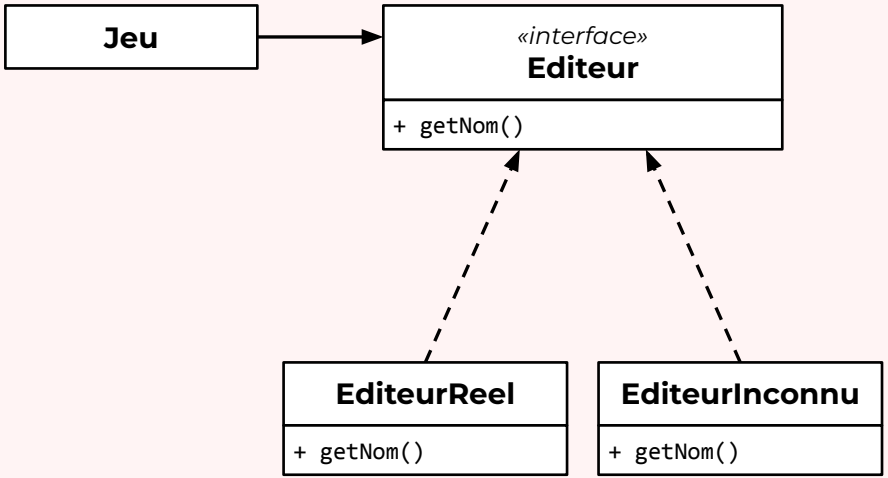
Nom générique	Rôle
AbstractObject	Contrat utilisé par le client (ici : Editeur)
RealObject	Implémentation réelle, avec le vrai comportement (ici : EditeurReel)
NullObject	Même contrat, comportement neutre et sans effet de bord (ici : EditeurInconnu)
Client	Utilise AbstractObject sans savoir quelle implémentation il manipule (ici : le code qui parcourt les jeux)

Diagramme générique vs notre exemple

Diagramme générique du Null Object



Notre exemple



Les limites du Null Object

Bugs masqués
L'absence devient silencieuse, alors qu'une exception aurait peut-être été préférable.

Pas toujours de comportement neutre
Que renvoyer pour un prix inconnu ? 0 serait faux.

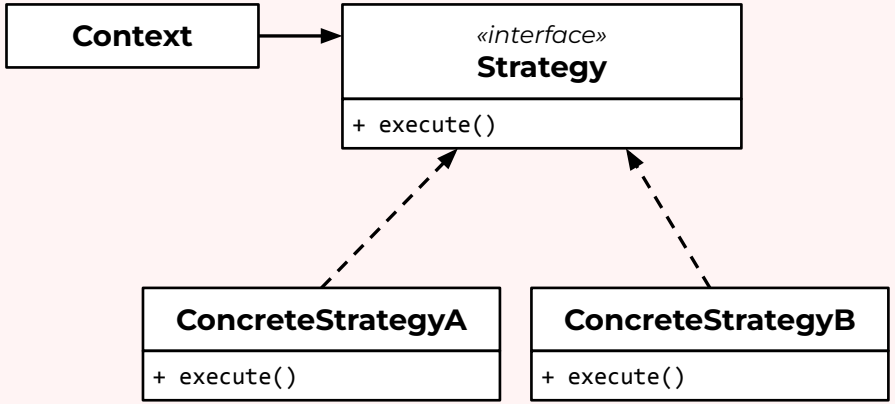
Plus de classes
Une interface et une classe par cas à écrire et à maintenir.

null renvoyé ailleurs
L'intérêt disparaît si une partie du code continue à renvoyer null.

Bonnes pratiques : une seule instance du Null Object (singleton, car sans état) ; éviter isNull() qui réintroduit un test.

Null Object et Stratégie

Intention : encapsuler une famille d'algorithmes interchangeables pour pouvoir en changer sans modifier le client.



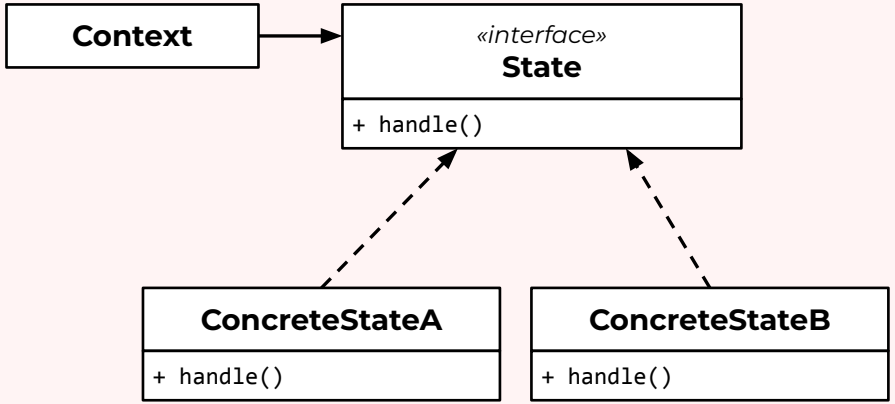
Ce qui rapproche : même structure, une interface et des implémentations substituables.

Lien : un Null Object est souvent une stratégie qui ne fait rien (ex. FlyNoWay dans les canards de R2.01).

Différence : Stratégie sert à choisir un comportement, Null Object à remplacer l'absence d'objet.

Null Object et État

Intention : permettre à un objet de modifier son comportement quand son état interne change.



Ce qui rapproche : même structure, une interface et des implémentations substituables.

Lien : un Null Object peut être vu comme un état « vide » ou « inactif » (ex. un lecteur sans morceau chargé).

Différence : dans État, les états changent au cours du temps ; le Null Object est généralement fixe.

Quelle solution choisir ?

Optional : signale l'absence et oblige l'appelant à la traiter.

Null Object : cache l'absence derrière un comportement par défaut.

requireNonNull : interdit null : on échoue tout de suite (fail-fast).

if / try-catch : cas simples ou code existant.

Pas de solution universelle : le choix dépend du contexte.

Live coding : le contexte

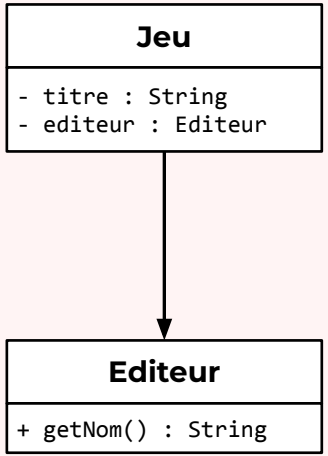
Contexte : application de ventes de jeux vidéo ; certains jeux n'ont pas d'éditeur.

Point de départ : du code avec null qui lève une NullPointerException.

Étapes :

- Interface Editeur
- EditeurReel et EditeurInconnu
- Chargement du CSV : EditeurInconnu si champ vide
- Suppression des if, test JUnit

Point de départ



Rétro-conception du live coding

Rétro-conception de notre code

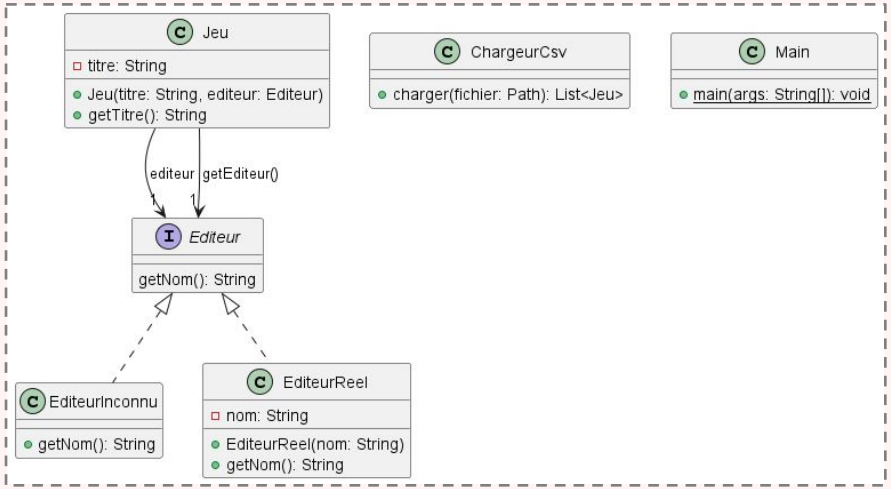
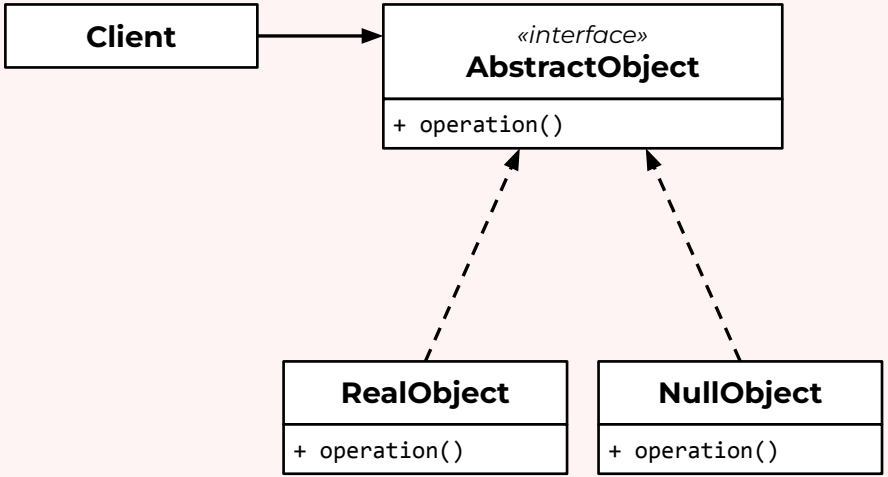


Diagramme générique du Null Object



LIVE CODING

URL : https://youtu.be/rEg9EO1hnak

QCM

Mise en pratique

SOURCES ET IAGRAPHIE

PARTIE SACHA

Sources & Bibliographie

Supports académiques :

TP d'introduction aux exceptions (IUT de Limoges, Isabelle Blasquez).

TP de prise en main de Mockito et du Kata *Game of Dices* (IUT de Limoges, Isabelle Blasquez).

Documentation officielle Java :

Oracle Documentation - *Class NullPointerException*.

Oracle Documentation - *Class Optional*.

Oracle Tutorial - *Throwing Exceptions*.

Articles et analyses techniques :

Développez.com - *Apprendre à utiliser la classe Optional pour éviter d'utiliser explicitement null*.

Le Touilleur Express (Nicolas Martignole) - *Optional en Java 8*.

Ressources encyclopédiques :

Wikipédia - *Null pointer*. Wikipédia - *Tony Hoare*.

IAgraphie (Gemini 3.5 Flash-Lite)

Génération de contenu : Rédaction d'une première ébauche de l'oral sur les thèmes abordés avec citation de sources (utilisation de documentation officiel, sources populaire type wikipedia et concepts TP) et exemples.

Structuration : Aide à l'organisation de la présentation et de la mise en forme du contenu pour les diapositives (slides).

Correction et révision : Clarification technique, correction de l'oral et des diapositives

SOURCES ET IAGRAPHIE

Partie Nathan

Sources :

- Wikipédia : Null object pattern
- Refactoring Guru : Introduce Null Object
- Martin Fowler : Special Case (catalogue PoEAA)
- Cours R2.01 et R3.04

IAgraphie (Claude) :

- Structuration du contenu sur Null Object
- Exemple Jeu / Editeur

SOURCES ET IAGRAPHIE

Partie Yassine

Sources :

- TP de prise en main de Mockito et du Kata Game of Dices (IUT de Limoges, Isabelle Blasquez)
- Oracle Documentation : Class Optional
- Oracle Documentation : Class Objects (méthode requireNonNull)
- Oracle Documentation : Record Classes
- Cours de Java de l'IUT : notion de record

IAgraphie :

- Claude (Anthropic) : mise en forme des diapositives de la partie Yassine
- Captures d'écran : exemples de code (Kata Dice, requireNonNull, record)